

Reinforcement Learning

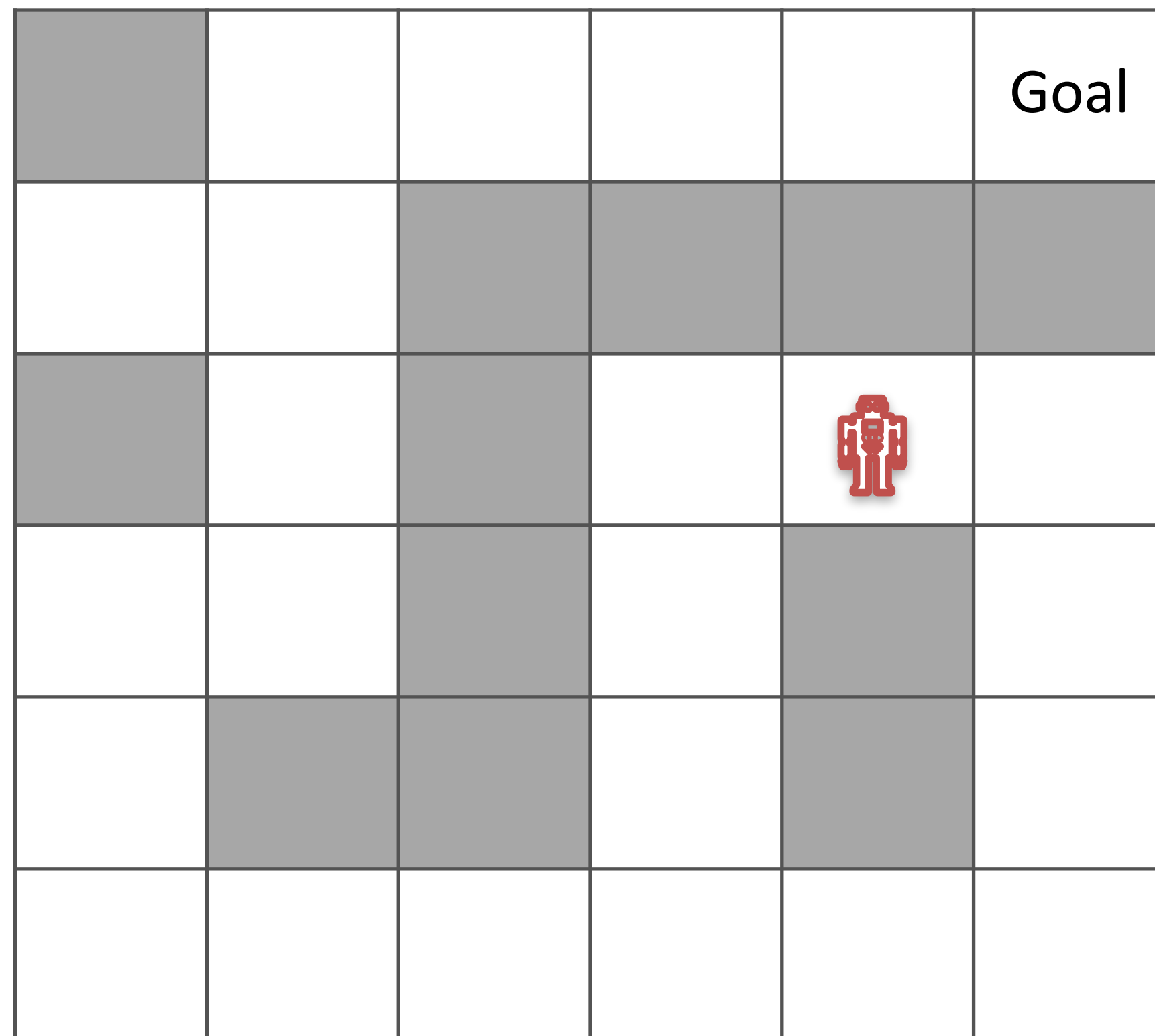
Markov Decision Process

Debapriyo Majumdar
Indian Statistical Institute
debapriyo@isical.ac.in

Beyond multi-arm bandits

- Multi-arm bandits: the task is to choose the best action
 - Stationary: find the single best action
 - Non-stationary: track the best action as it changes over time
 - The situation (context) does not change
- Towards more realistic setting
 - More than one situation

Example 1: A Gridworld

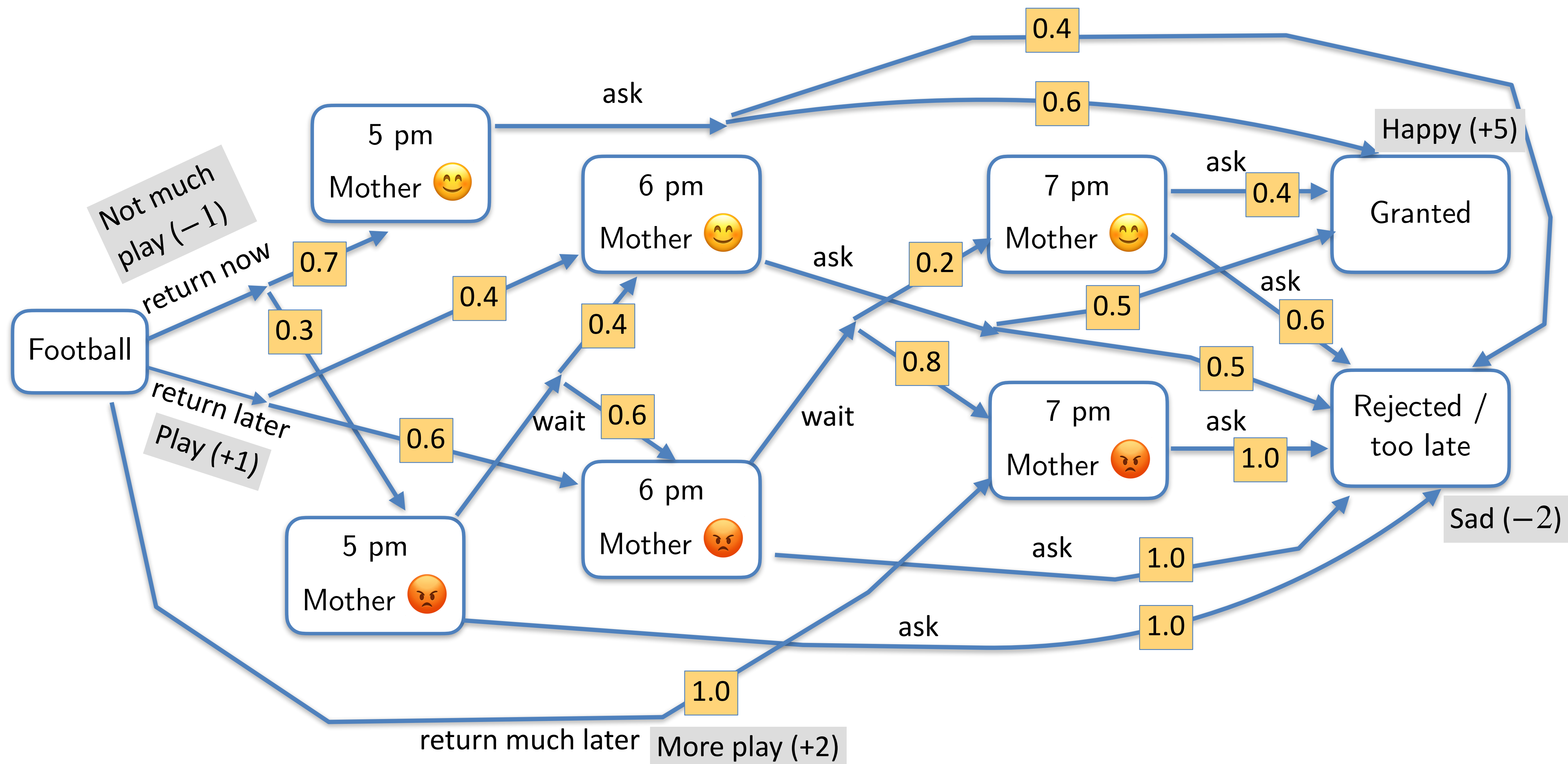


Observation

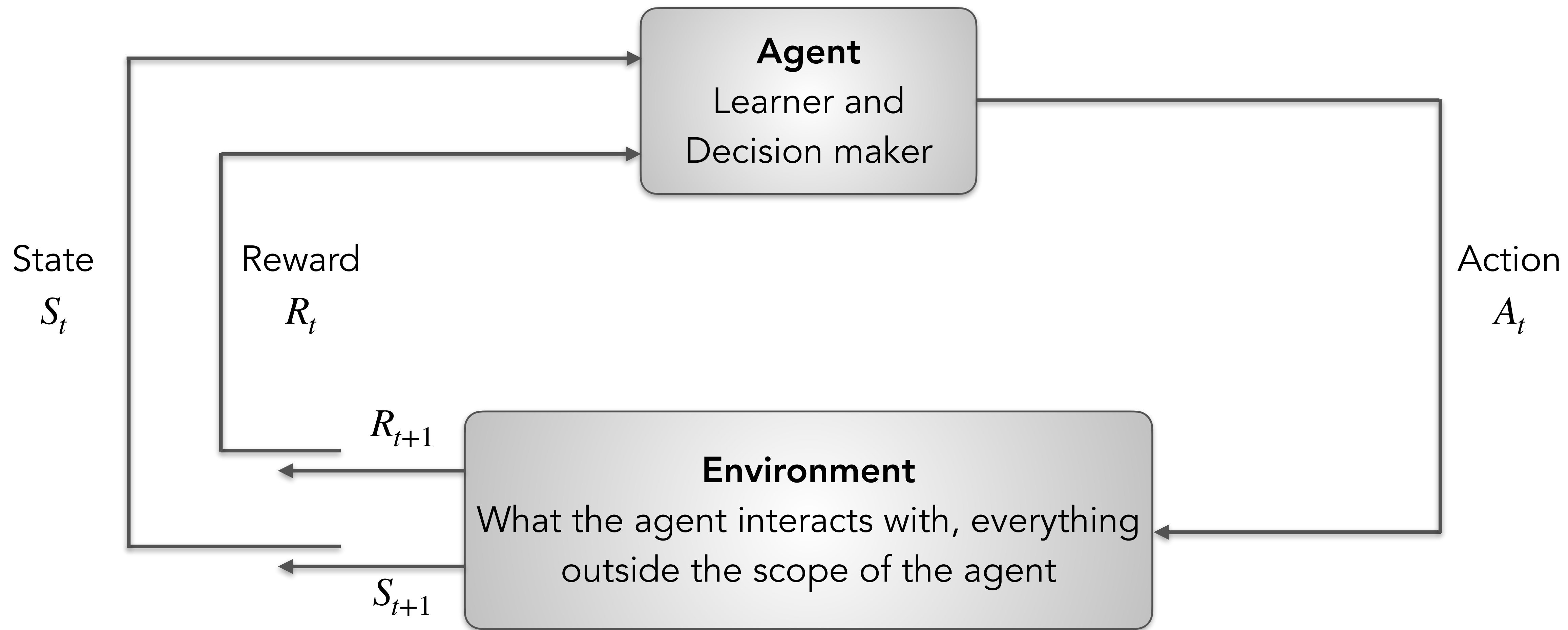
There is no uncertainty about the next position given the current position and the action

- A robot that has no visual sensor
- Needs to figure out the shortest path to the goal in a grid with obstacles
- Action: At each step, can try to go north, south, east or west
- What happens?
 - May hit a wall (stays at the same position)
 - Move to an adjacent cell
- What are the rewards?
 - Stay at a the same cell: wasted one move, may also consider slight penalty for hitting
 - Move to adjacent cell: made the move count
 - More later on how to model this right

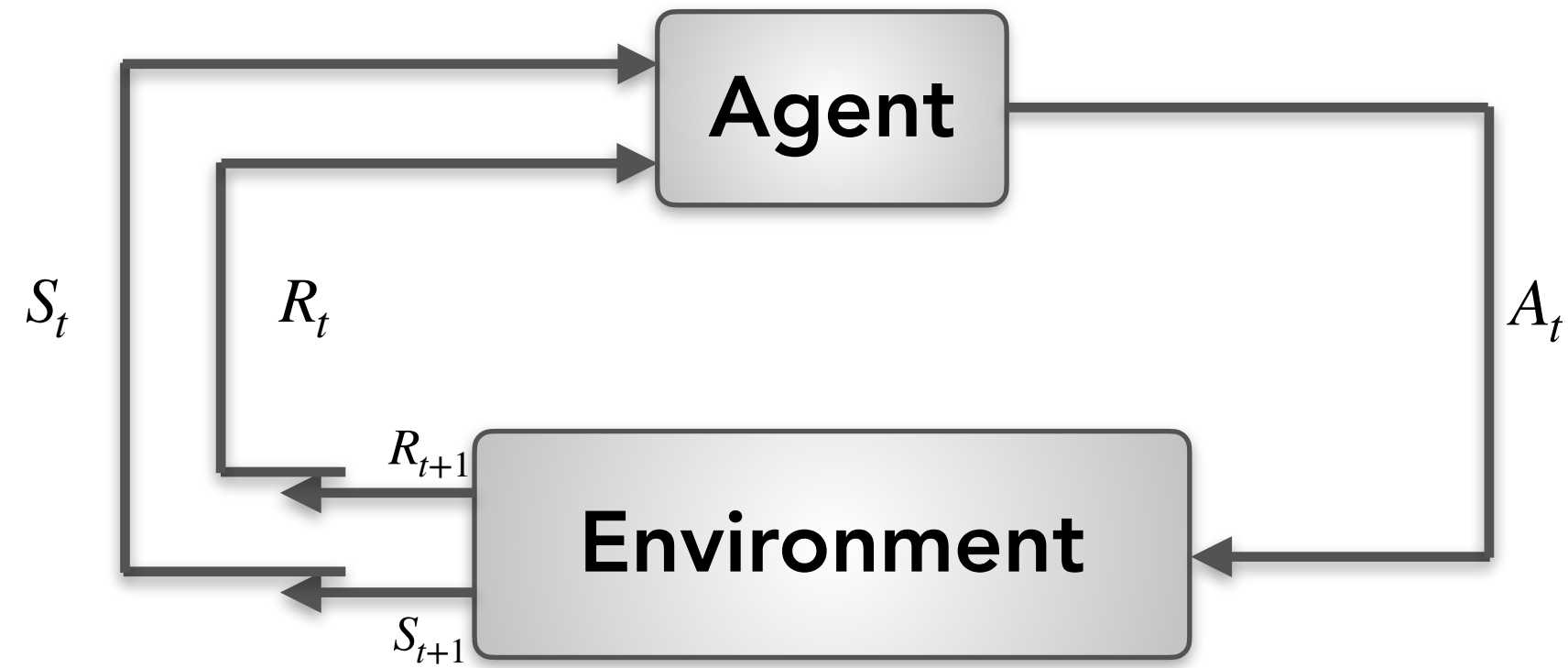
Example 2: Asking Permission for Sleepover at Friends'



The agent - environment interface



Finite MDP



$\mathcal{S}$ = set of states

$\mathcal{R}$ = set of rewards

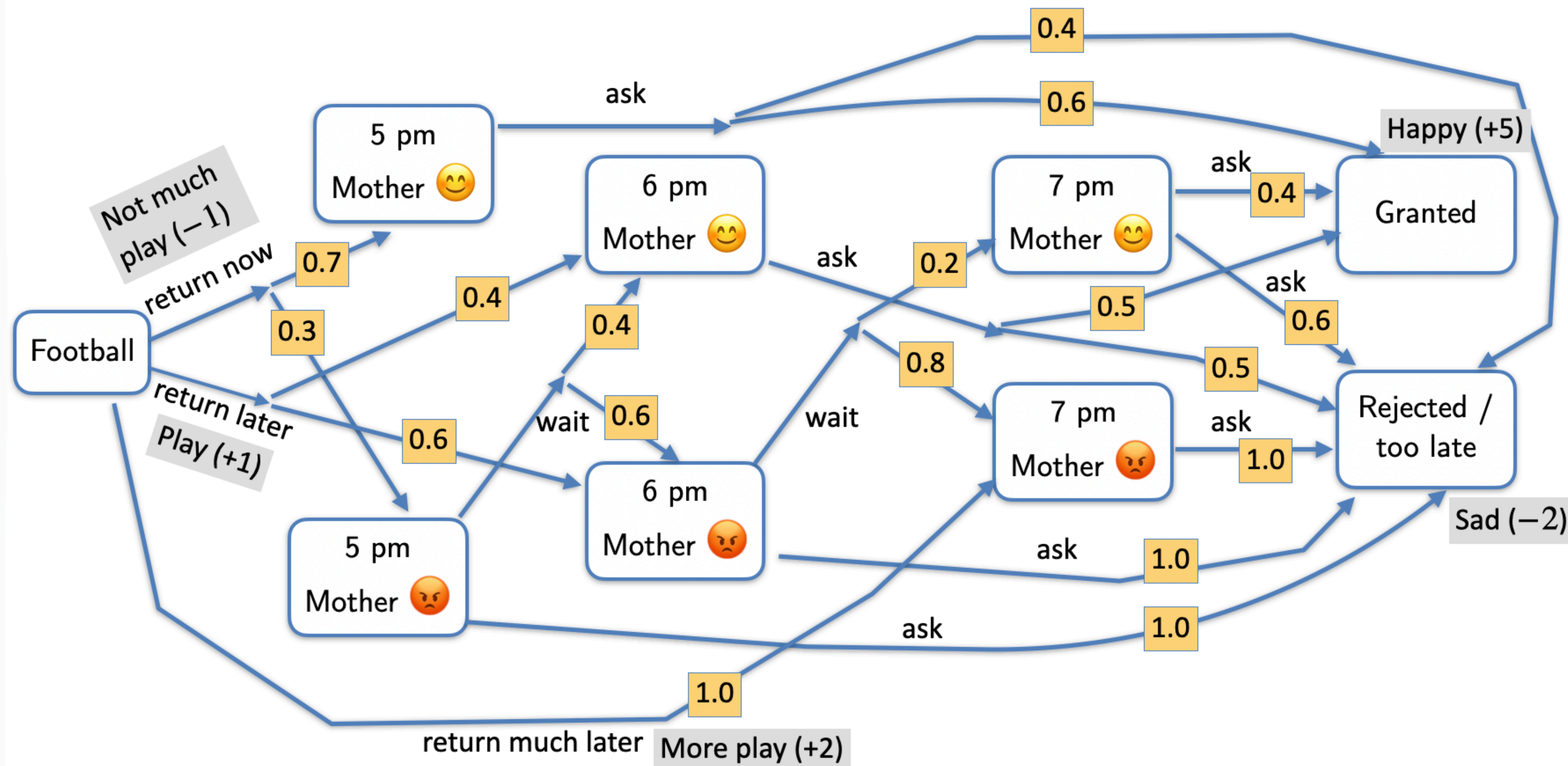
$\mathcal{A}$ = set of actions

All are finite sets

- Sequence (trajectory) of actions, rewards and states:
 $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots$
- For finite MDP, we have a probability distribution
 $p(s', r | s, a) \doteq \mathbb{P}[S_t = s', R_t = r | S_{t-1} = s, A_{t-1} = a]$ for all
 $s, s' \in \mathcal{S}, a \in \mathcal{A}$ and $r \in \mathcal{R}$
- Also, we have

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) = 1, \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}$$
- Markov property: the probabilities of each possible value of S_t and R_t depends only on S_{t-1} and A_{t-1} , not any state of action before that
- The state must contain all necessary information about the past that matters for the future

Example



Examples of the probability distribution p (in class)

Example: simplified recycling robot

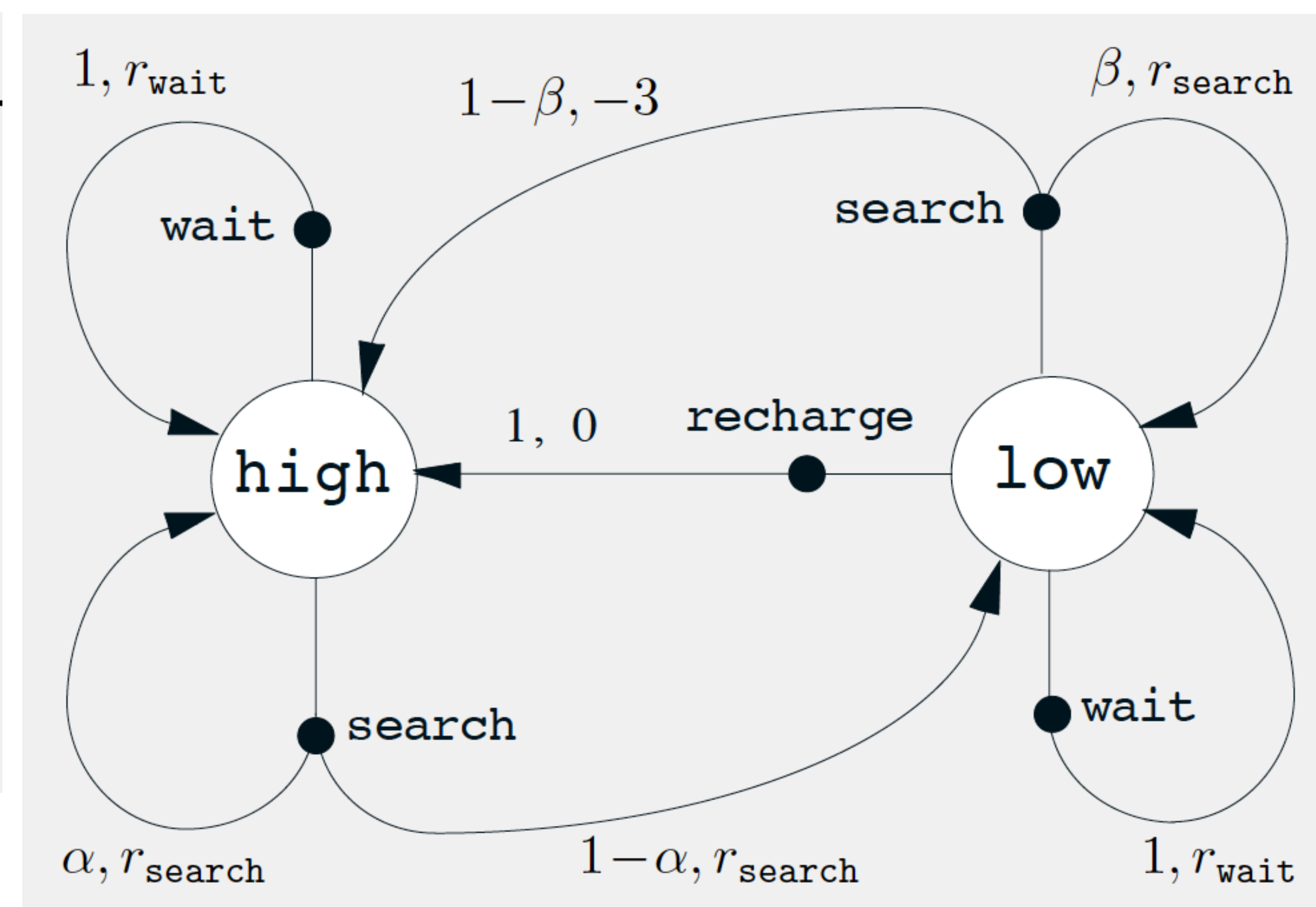
- Mobile robot: needs to collect empty soda cans in office and put them into the bin
 - Has all the necessary sensors and arms (not important in our discussion)
 - Runs on a rechargeable battery (but it better decide when to recharge itself)
- Two states indicating battery level { high, low }
- Three actions:
 - search: search for an empty can by moving around the office
 - wait: wait for someone to bring an empty can
 - recharge: go to the home base and get recharged
- State - action mapping:
 - $A(\text{low}) \in \{ \text{search}, \text{wait}, \text{recharge} \}$
 - $A(\text{high}) \in \{ \text{search}, \text{wait} \}$, since there is obviously no point in recharging

Recycling robot: rewards

- Positive reward for finding a can and dumping it into the bin
- High negative for battery going down (needs to be rescued manually)
- search: chance of reward r_{search} higher than others, but drains battery faster

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-

The probability distribution



Transition graph

Goals and rewards

- Rewards: signal whether the agent is making progress towards the goal or not
 - However, needs to be formulated carefully
 - The notion of maximizing long term rewards should coincide with achieving the goal
- Example 1: make a robot learn to escape from a maze
 - Reward -1 for every passing time-step, $+1$ when escaped (but not otherwise)
- Example 2: make a machine learn to play chess
 - Reward $+1$ for winning, -1 for losing, 0 for a draw
 - What if we set rewards based on subgoals?
 - Suppose: $+1$ for taking a piece of the opponent, -1 for losing a piece
 - But in a situation where it's important to save the king, the agent may take another piece of the opponent and lose
 - Or, never go for a checkmate

Episodic tasks

- Informally, the goal is to maximize the cumulative reward in the long run
- Episodic task: one with a natural notion of a final or terminal state
 - Plays of a game (ends when the game ends): one episode
 - A robot escapes the maze
- The learning does not finish, but once a game is finished, the next game start (fresh)
 - The state is reset
- If current time-step is t , then we want to maximize expected return

$$G_t = R_{t+1} + R_{t+2} + \cdots + R_T$$

where T is the time of termination (a random variable) varies from episodes to episodes

Continuing tasks and discounting

- No identifiable episodes, rather goes on
 - On-going process control task
- There is no terminal state, or it can be thought of as $T = \infty$
- The reward also can be infinity
- Discounting: reduce weights of rewards in the future exponentially

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where $\gamma \in [0,1]$ is called the discount rate (a parameter)

Discounting: intuition

- The discount rate γ determines how much into the future the agent is focussed on

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- If $\gamma = 0$, $G_t = R_{t+1}$, focussing on only the next reward

- Analogy: instant gratification

- More the γ , more futuristic the agent is

- For any $\gamma < 1$, the infinite sum converges if the rewards are bounded by some number R , because the sum of the weights converges: $\sum_{k=0}^{\infty} \gamma^k = \frac{1}{1-\gamma}$

- Relationship between (expected) returns of successive time steps:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) = R_{t+1} + \gamma G_{t+1}$$

Policy, Value Function and Action-Value Function

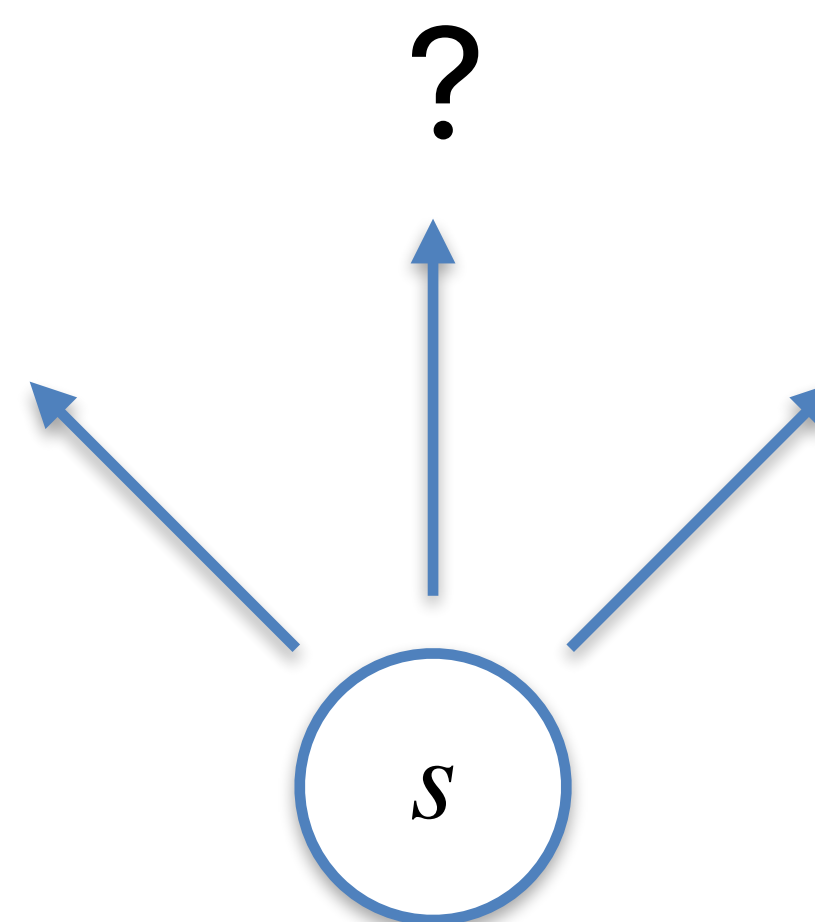
Now that I am in this situation, what should I do?

Policy: mapping from states to probabilities of selecting an action

$$\pi(a \mid s) \text{ for all } s \in \mathcal{S}$$

Value: If I follow policy π from state s , what will be my ultimate (expected) reward?

$$\begin{aligned} v_{\pi}(s) &= \mathbb{E}_{\pi} [G_t \mid S_t = s] \\ &= \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right] \end{aligned}$$



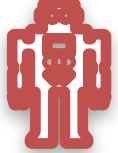
Action-Value: From state s , if I take action a first and then follow policy π from state s , what will be my ultimate (expected) reward?

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} [G_t \mid S_t = s, A_t = a] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

Example

Suppose

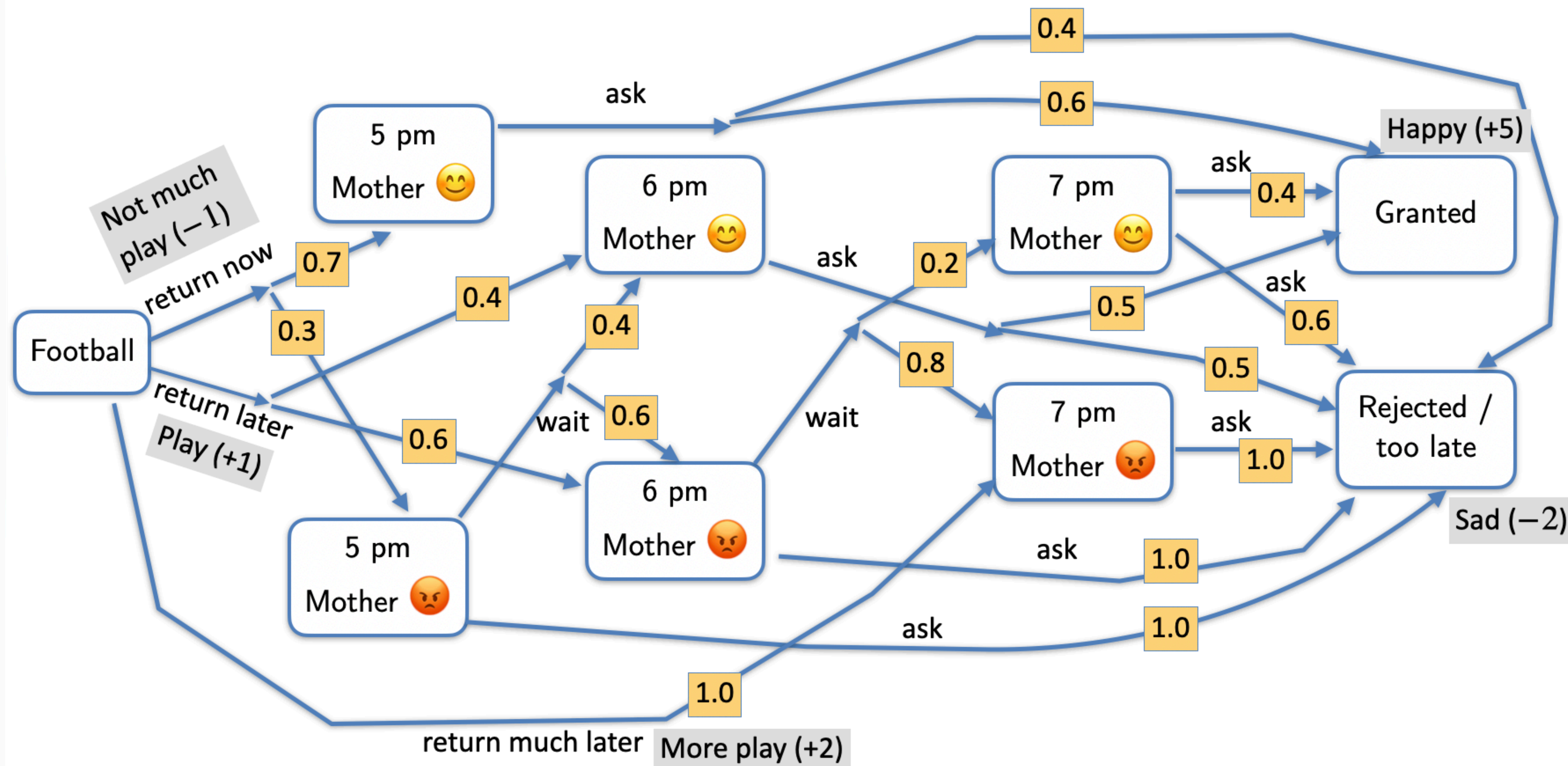
- Let π be the policy such that $\pi(s, \text{up}) = \pi(s, \text{right}) = 1/2$ for all states s .
- What is $v_\pi(A)$?
- What is $q_\pi(A, \text{up})$?
- Try a few more of your choice ...

		C		A	Goal
					B

Reward for:

- Reaching the goal: 0
- Reaching any other cell: -1
- Stays at the same place if it hits the wall

Example



- Poli
- $v_{\pi}(\text{Football}) = ?$
- $v_{\pi}(\text{6 pm - Mother happy}) = ?$
- $v_{\pi}(\text{Football}) = ?$
- $v_{\pi}(\text{5 pm - Mother unhappy}) = ?$

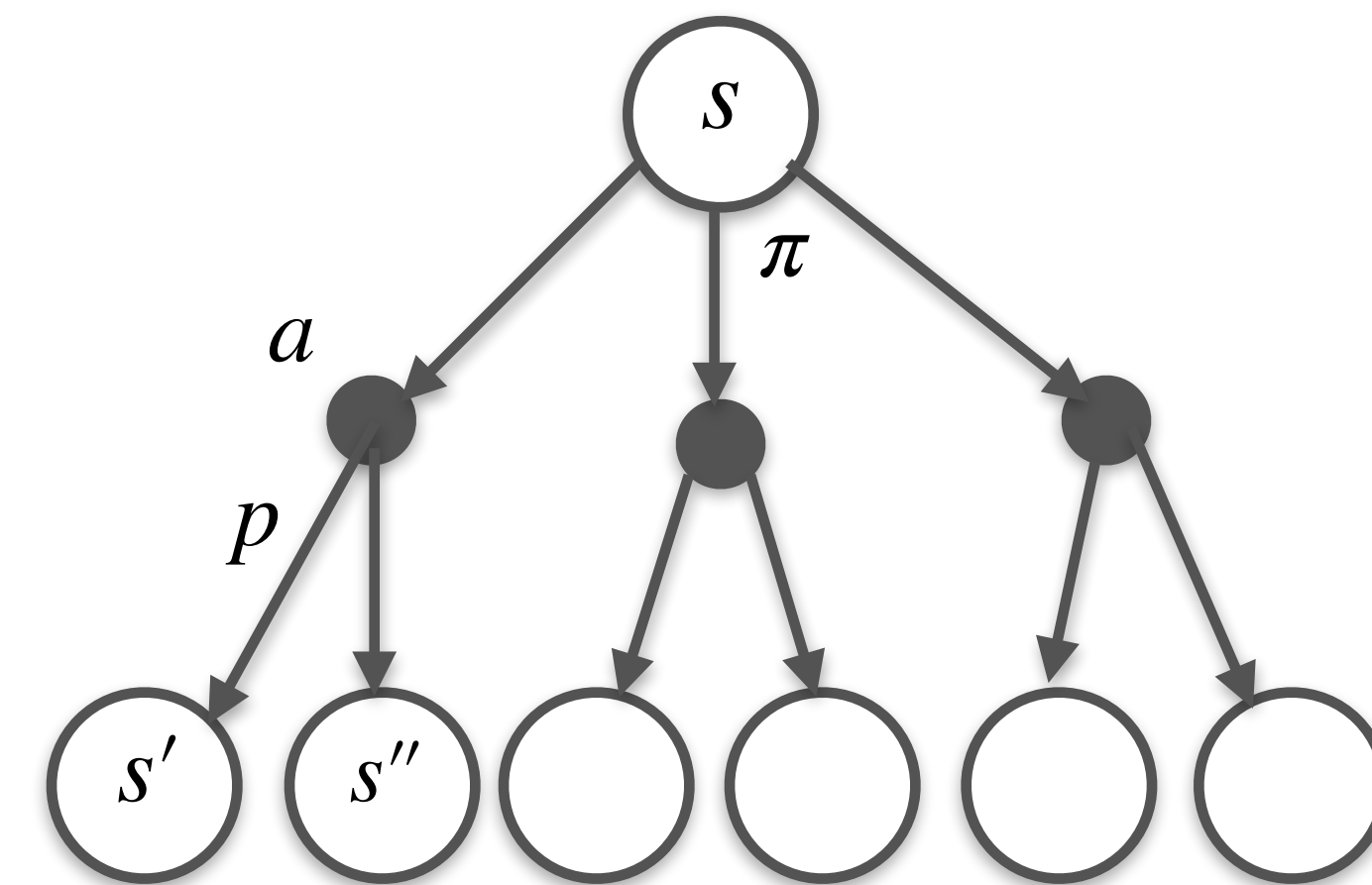
Bellman equation

- From the definition of value function, we have

$$\begin{aligned}
 v_{\pi}(s) &= \mathbb{E}_{\pi} \left[G_t \mid S_t = s \right] \\
 &= \mathbb{E}_{\pi} \left[R_{t+1} + \gamma G_{t+1} \mid S_t = s \right] \quad (\text{since } G_t = R_{t+1} + \gamma G_{t+1}) \\
 &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \left[r + \gamma \mathbb{E}_{\pi} \left[G_{t+1} \mid S_{t+1} = s' \right] \right] \\
 &= \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma v_{\pi}(s') \right]
 \end{aligned}$$

- A relationship between *value* of a state s and the *values* of its successor states s'
- Average over all possible actions, weighting them according to their probabilities and all possible subsequent states and rewards

Backup diagram for v_{π}



Optimal policy

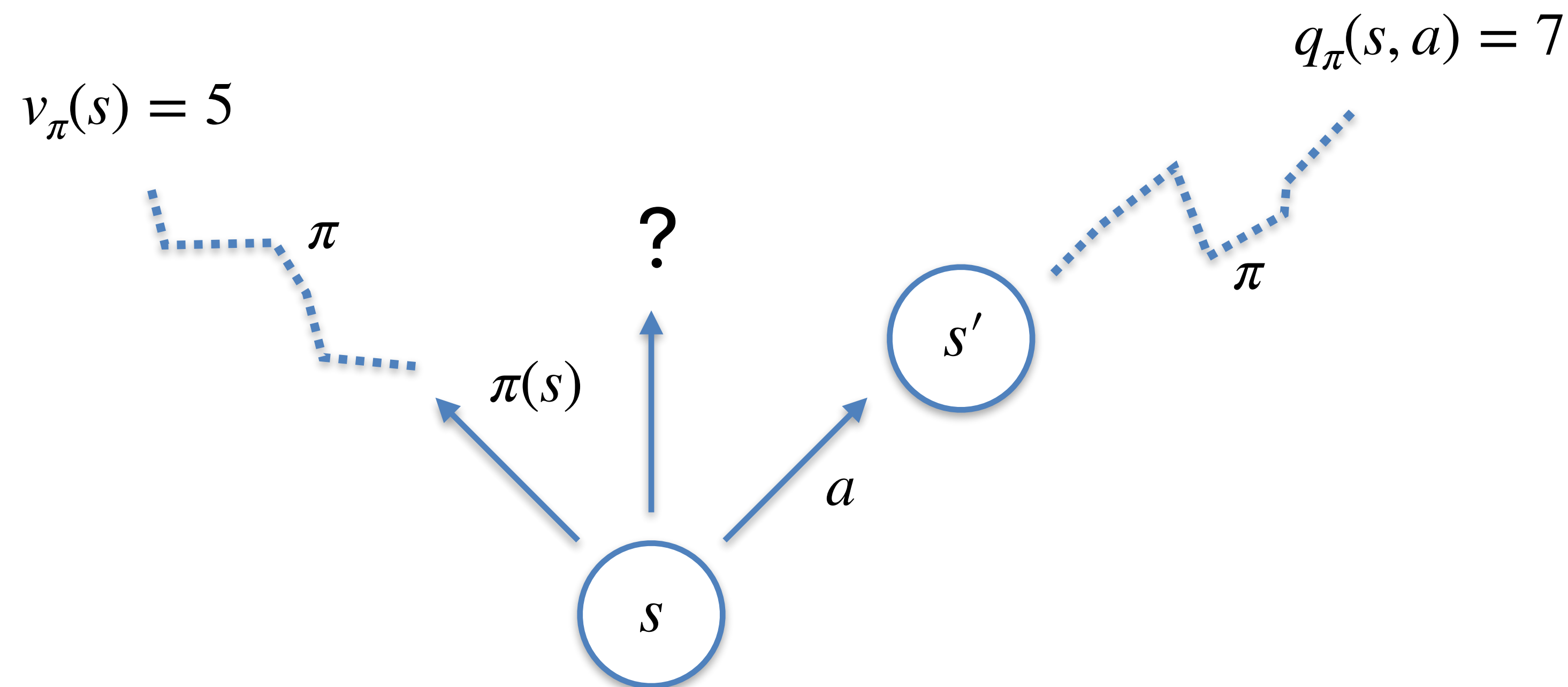
- Goal of a “good” policy: accumulate a lot of reward in the long run
 - Formally, the value function should have high values
- Consider two policies π and π'
 - The policies are defined by their probabilities of choosing actions given a state
 - It is possible that at some state π chooses a better action, in some other state otherwise
- Can define a partial order between policies
 - Say: $\pi \geq \pi'$ if and only if $v_\pi(s) \geq v_{\pi'}(s)$ for all states $s \in \mathcal{S}$
- Optimal policy π_* : one which is better or equal to all other policies
- Can be more than one optimal policies
 - Exercise: find an example of that
- However, they must have the same value function
 - If the value functions differ, we can have an optimal policy with the higher values at each state

Optimal state-value function

- Optimal state-value function: the value function v_* of the optimal policies
$$v_*(s) = \max_{\pi} v_{\pi}(s) \text{ for all } s \in \mathcal{S}$$
- Optimal action-value function: action value function of the optimal policies
$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a) \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}(s)$$
- Intuition: from state s , take action a and follow an optimal policy thereafter
- We can also write:
- $$q_*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a \right]$$

Optimal Policy

Optimal Policy: the policy which is expected to give best return from all states



Instead of taking the action according to my policy π , it would have been better to take action a

Policy improvement (control): Can change $\pi(s)$ to a

Optimal policy is one which cannot be improved any further

Bellman optimality equations

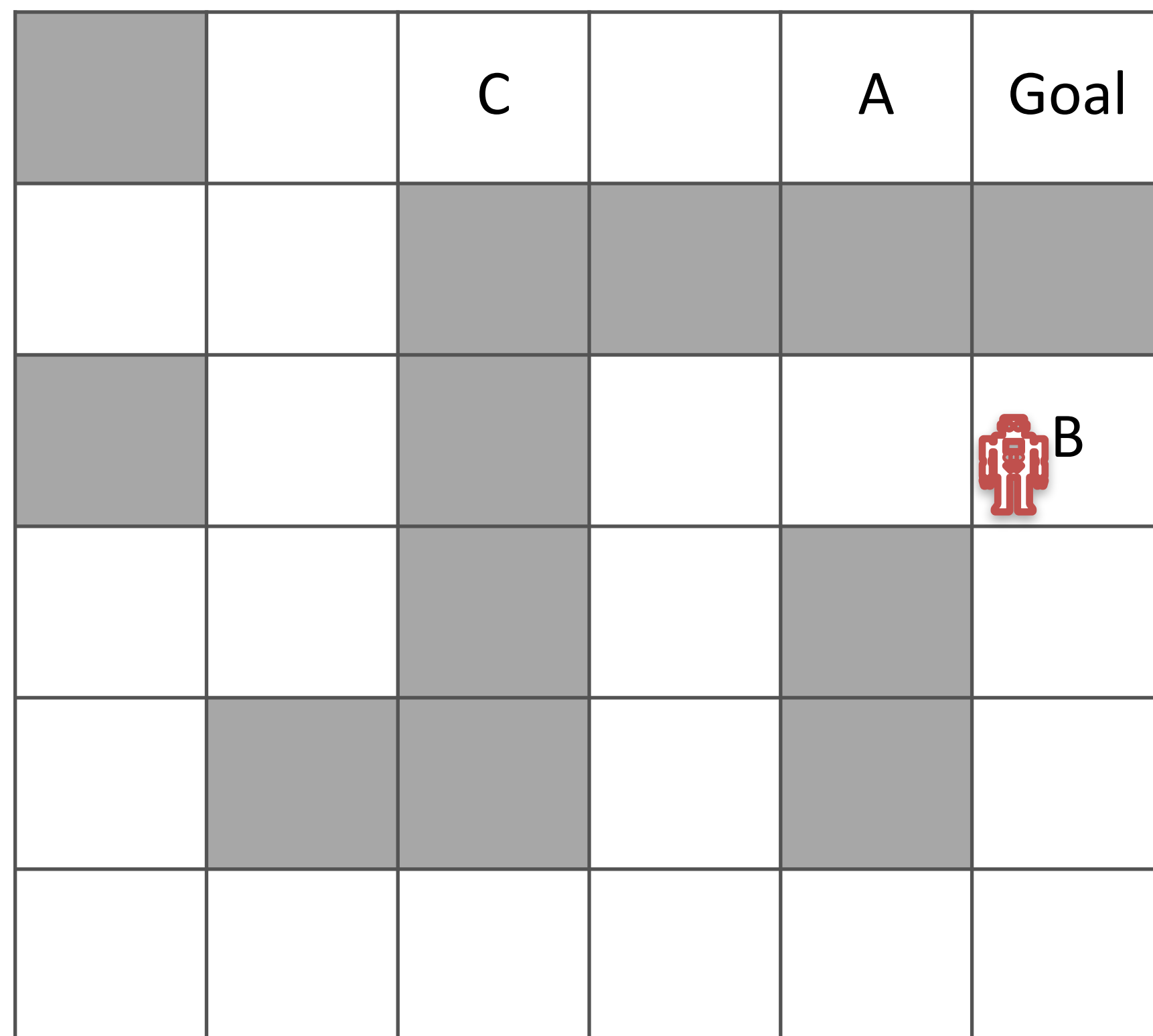
- Bellman optimality equation: Bellman equation for the optimal value function
- Consider the agent at some state
- The value of π_* must be the expected return for the best action from thereon

$$\begin{aligned}
 \blacksquare \quad v_*(s) &= \max_{a \in \mathcal{A}} q_{\pi_*}(s, a) \\
 &= \max_a \mathbb{E}_{\pi_*} \left[G_t \mid S_t = s, A_t = a \right] \\
 &= \max_a \mathbb{E}_{\pi_*} \left[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a \right] \\
 &= \max_a \mathbb{E} \left[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a \right] \\
 &= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')]
 \end{aligned}$$

Example

Exercises:

- $v_*(A) = ?$
- $q_*(A, \text{up}) = ?$
- $v_*(B) = ?$
- $q_*(B, \text{down}) = ?$
- Is the optimal policy unique?
- What if the rewards are changed as:
 - Reward for reaching the goal is 1
 - Reward for reaching any other state is 0



Reward for:

- Reaching the goal: 0
- Reaching any other cell: -1
- Stays at the same place if it hits the wall

Recall: simplified recycling robot

- Mobile robot: needs to collect empty soda cans in office and put them into the bin
 - Has all the necessary sensors and arms (not important in our discussion)
 - Runs on a rechargeable battery (but it better decide when to recharge itself)
- Two states indicating battery level { high, low }
- Three actions:
 - search: search for an empty can by moving around the office
 - wait: wait for someone to bring an empty can
 - recharge: go to the home base and get recharged
- State - action mapping:
 - $A(\text{low}) \in \{ \text{search}, \text{wait}, \text{recharge} \}$
 - $A(\text{high}) \in \{ \text{search}, \text{wait} \}$, since there is obviously no point in recharging

Bellman optimality equations for the recycling robot

$$\begin{aligned}
 v_*(h) &= \max \begin{cases} p(h|h, s)[r(h, s, h) + \gamma v_*(h)] + p(1|h, s)[r(h, s, 1) + \gamma v_*(1)] \\ p(h|h, w)[r(h, w, h) + \gamma v_*(h)] + p(1|h, w)[r(h, w, 1) + \gamma v_*(1)] \end{cases} \\
 &= \max \begin{cases} \alpha[r_s + \gamma v_*(h)] + (1 - \alpha)[r_s + \gamma v_*(1)] \\ 1[r_w + \gamma v_*(h)] + 0 \end{cases} & \begin{array}{l} h = \text{high}, 1 = \text{low} \\ s = \text{search}, w = \text{wait}, r = \text{recharge} \end{array} \\
 &= \max \begin{cases} r_s + \gamma[\alpha v_*(h) + (1 - \alpha)v_*(1)] \\ r_w + \gamma v_*(h) \end{cases}
 \end{aligned}$$

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-

$$v_*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')]$$

Bellman optimality equations for the recycling robot

$$\begin{aligned}
 v_*(1) &= \max \begin{cases} p(h | 1, s) [r(1, s, h) + \gamma v_*(h)] + p(1 | 1, s) [r(1, s, 1) + \gamma v_*(1)], \\ p(h | 1, w) [r(1, w, h) + \gamma v_*(h)] + p(1 | 1, w) [r(1, w, 1) + \gamma v_*(1)], \\ p(h | 1, r) [r(1, r, h) + \gamma v_*(h)] + p(1 | 1, r) [r(1, r, 1) + \gamma v_*(1)] \end{cases} \\
 &= \max \begin{cases} (1 - \beta) [-3 + \gamma v_*(h)] + \beta [r_s + \gamma v_*(1)], \\ 0 + 1 [r_w + \gamma v_*(1)], \\ 1 [0 + \gamma v_*(h)] + 0 \end{cases}
 \end{aligned}$$

$h = \text{high}, 1 = \text{low}$
 $s = \text{search}, w = \text{wait}, r = \text{recharge}$

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-

$$\begin{aligned}
 v_*(s) &= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')] \\
 &= \max \begin{cases} \beta r_s - 3(1 - \beta) + \gamma [(1 - \beta)v_*(h) + \beta v_*(1)], \\ r_w + \gamma v_*(1), \\ \gamma v_*(h) \end{cases}
 \end{aligned}$$

Bellman optimality equations for the recycling robot

- So, we have:

$$v_*(h) = \max \begin{cases} r_S + \gamma[\alpha v_*(h) + (1 - \alpha)v_*(1)] \\ r_W + \gamma v_*(h) \end{cases}$$

$$v_*(1) = \max \begin{cases} \beta r_S - 3(1 - \beta) + \gamma [(1 - \beta)v_*(h) + \beta v_*(1)], \\ r_W + \gamma v_*(1), \\ \gamma v_*(h) \end{cases}$$

- For any particular choice of r_S, r_W and α, β, γ , we have two non-linear equations with two unknowns, namely $v_*(h)$ and $v_*(1)$

Discussion

- One approach to find an optimal policy: solve the Bellman equations
- Problems:
 - We need to know the exact dynamics of the environment, i.e., the function p
 - Assumption of Markov property
 - Often, prohibitively expensive computational requirement

References

- Sutton, R.S. and Barto, A.G., 2018. Reinforcement learning: An introduction. MIT press.
<http://incompleteideas.net/book/the-book.html> (primary reference), Chapter 3